CHAR-Adapt-R3-BD-JIS-R++ (D3C-OM2)

A formal specification + implementation sketch for **character-biased attention with in-training causal switchbacks**, **theory-matched nulls**, and **valid inference**.

This document consolidates the development path:

- Phase-coded, homomorphism-preserving character masks on $\mathbb{F}_p^\times$
- ST-Gumbel hard routing over character orders $k \mid (p-1)$
- Joint bandit (instrumented) over $(\tau, \beta, \text{type})$
- Switchback probes: within-weight counterfactual deltas Δ_t
- Null ladder with **character-sampled nulls** (N3) using **row-mean matching** and **norm matching**
- Primary endpoint + block-bootstrap CI; secondary endpoints Holm-gated

---

1. Core objects

1.1 Prime and index sets

- Choose prime $p \geq n$ per batch from a curriculum $\mathcal{P}$.
- Work on indices $i, j \in \{0, \ldots, n-1\} \subset \mathbb{F}_p$.
- Multiplicative group $G = \mathbb{F}_p^\times$ has order $|G| = p - 1$.

1.2 Allowed character orders

$$ \mathcal{K}(p)=\{k: k\mid(p-1),\ k\le k_{\max}\}. $$

1.3 Coupling map

We need a map $f(i,j) \in \mathbb{F}_p$ that produces a group element when possible.

Multiplicative coupling (preferred for multiplicative tasks):

- If $j \neq 0$: $f(i,j) = i \cdot j^{-1} \pmod{p}$.
- If $j = 0$: map to a special class ∞ (explicit zero-handling).
- Optionally distinguish $i = 0$ (class "0").

Rationale: avoid ad hoc hacks; treat zeros as explicit mask-vocabulary classes.

---

2. Phase-coded character mask construction

(Background: character theory and orthogonality follow classical finite-field treatments; see Serre, 1997. The use of characters as inductive bias contrasts with representation-based equivariance in Cohen & Welling, 2016; Kondor & Trivedi, 2018.)

To compare across primes, define a canonical primitive root g of $\mathbb{F}_p^\times$, e.g. the **smallest** primitive root.

For each $k \mid (p-1)$, the subgroup of k-th roots of unity has generator

$$ q_k = g^{(p-1)/k} \in \mu_k\subset G. $$

Index the subgroup elements as q_k^m for $m = 0, \ldots, k-1$. Define

$$ \iota_k(q_k^m)=m. $$

For $x \in G$, define

$$ r_k(x)=x^{(p-1)/k}\in \mu_k. $$

2.3 Phase-coded bias (homomorphism-preserving)

Given $m = \iota_k(r_k(x))$, define

$$ b_k(x)= a_k\cos\Big(\tfrac{2\pi m}{k}\Big) + a'_k\sin\Big(\tfrac{2\pi m}{k}\Big). $$

Often set $a'_k = 0$ for simplicity; keep a_k learnable per head or per layer.

2.4 Mask matrix

Define raw mask $\tilde{M}^{(k)} \in \mathbb{R}^{n\times n}$:

$$ \tilde M^{(k)}_{ij}= \begin{cases} b_k(f(i,j)) & j\ne 0\\ b & j=0 \end{cases} $$

with explicit constants/learned scalars for zero-classes.

Use a **fixed, shared** normalization for both structured and null masks:

- Center: $M \leftarrow M - \mathrm{mean}(M)$
- Scale: $M \leftarrow M/(\mathrm{std}(M) + \epsilon)$
- Optional row-centering: $M_{i\cdot} \leftarrow M_{i\cdot} - \overline{M_{i\cdot}}$

Denote the normalized mask $M^{(k)} = \psi(\tilde{M}^{(k)})$.

---

3. Attention with hard routing

(Context: hard routing via ST-Gumbel relates to discrete latent-variable training and mixture-of-experts, but here routing is used to preserve algebraic purity rather than efficiency; cf. Yun et al., 2020 for expressivity motivations.)

For head h:

$$ S_h=\frac{Q_hK_h^\top}{\sqrt{d_h}}. $$

Sample an order $k_h \in \mathcal{K}(p)$ using hierarchical ST-Gumbel with temperature τ:

$$ k_h \sim \mathrm{ST\text{-}Gumbel}(\pi_h,\tau). $$

Logits with mask strength β:

$$ \tilde S_h = S_h + \beta M^{(k_h)},\quad A_h=\mathrm{softmax}(\tilde S_h). $$

Regularizers (typical):

- Entropy band on A_h
- Lipschitz/slope proxy penalty
- Budget on expected complexity $\mathbb{E}[\mathrm{complexity}(k_h)]$
- (Optional) diagnostic-only normality shaping applied late, small weight

---

4. Null ladder and switchback probes

(Methodological context: switchback-style counterfactual evaluation is inspired by online experimentation and sequential testing; see Efron & Tibshirani, 1994; Lahiri, 2003 for resampling under dependence, and Popper, 1959 for falsification-first design.)

4.1 Null ladder

- **N1 (weak null):** class-preserving shuffle of phase labels (distribution matched)
- **N2 (stronger):** N1 + row-mean matching
- **N3 (theory-matched null):** sample $\ell \neq k$ from other character orders, build $M^{(\ell)}$, apply same ψ, then row-mean match + **norm-match**

4.2 Row-mean matching (cheap, artifact-avoiding)

For any candidate null M^{null} and target structured $M^{(k)}$:

$$ \widehat{M}^\text{null}_{i\cdot}=M^\text{null}_{i\cdot}-\overline{M^\text{null}_{i\cdot}}+\overline{M^{(k)}}. $$

4.3 Norm matching (defensive against "tuned null" critiques)

After row-mean matching:

$$ \widehat{M}^\text{null} \leftarrow \widehat{M}^\text{null}\cdot \frac{|M^{(k)}|_F}{|\widehat{M}^\text{null}|_F+\epsilon}. $$

4.4 Orthogonality correlation (logged, not hard-excluded)

Define

$$ c_{k\ell}=\frac{\langle M^{(k)},\widehat{M}^{(\ell)}\rangle_F}{|M^{(k)}|_F|\widehat{M}^{(\ell)}|_F}. $$

Use $|c_{k\ell}|$ only to **weight** null sampling, not to exclude (prevents pool shrinkage).

4.5 Matched-null weighting (D3C-OM2)

Preferred scheme: **norm-match always**, then weight only by correlation:

$$ \Pr(\ell\mid k) \propto \max(w_{\min},\tfrac{1}{1+|c_{k\ell}|}). $$

Set $w_{\min} = 0.05$ (floor), then renormalize.

With weights fixed at W_t, compute forward-only evaluator $\widehat{R}_t$ on a fixed heldout minibatch (or fixed slice).

For N3:

$$ \Delta_t^{(N3)}=\widehat{R}_t(\text{structured})-\widehat{R}_t(\text{null N3}). $$

---

5. Controller: instrumented joint bandit (sparse arms)

(Controller background: EXP3 and related adversarial bandits follow Bubeck & Cesa-Bianchi, 2012; Shalev-Shwartz et al., 2012. Here the controller is used instrumentally to test causal relevance of ***eta>0****, not to maximize IID performance.)*

Action space (example, 9 arms):

- $\tau \in \{\tau_L, \tau_M, \tau_H\}$
- $\beta \in \{0, \beta_M, \beta_H\}$
- type $\in \{\text{structured}, \text{shuffle}\}$ **included as arms** or as internal switchback probes.

Use EXP3 with clipped reward $R_t \in [-0.5, 0.5]$, uniform initialization.

Instrumented reporting:

- Time-resolved posterior mass on $\beta = 0$ vs $\beta > 0$
- Time-resolved preference for structured vs shuffle arms
- Correlation between preference and Δ_t

---

6. Inference: primary endpoint + valid CI

(Statistical context: block bootstrap for time-dependent probes follows Lahiri, 2003; weighted medians as robust estimators follow classical order-statistics theory.)

6.1 Probe scheduling

- Hybrid: 70% uniform spacing + 30% ACF/variance-triggered, hard-capped budget.
- Log trigger reasons.

6.2 Primary endpoint (pre-registered)

Fix a post-warmup window $[t_0, t_1]$ (e.g., 30%–80% of training).

Compute probe-level variance $\widehat{\mathrm{Var}}(\Delta_t)$ via evaluator bootstrap or robust estimator.

Primary statistic:

- **Weighted median** of $\Delta_t^{(N3)}$ with weights $w_t \propto 1/(\widehat{\mathrm{Var}}(\Delta_t) + \epsilon)$.

6.3 CI via block bootstrap over probe times

Resample probe indices in contiguous blocks (block length chosen from empirical autocorrelation). Compute CI for the weighted median.

Secondary endpoints (AUC, Corr, etc.) are Holm-gated.

Code snippets (PyTorch-style)

> These are reference snippets meant to be pasted into an experiment repo. They omit some engineering details (device placement, caching, logging) for clarity.

A. Primitive root + factorization helpers (small p)

```python
import math

def prime_factors(n: int) -> list[int]:
    """Return unique prime factors of n (trial division; fine for n<1e6)."""
    fac = []
    x = n
    d = 2
    while d * d <= x:
        if x % d == 0:
            fac.append(d)
            while x % d == 0:
                x //= d
        d += 1 if d == 2 else 2
    if x > 1:
        fac.append(x)
    return fac

def is_primitive_root(g: int, p: int) -> bool:
    """Check if g is a primitive root mod p."""
    if g % p == 0:
        return False
    phi = p - 1
    fac = prime_factors(phi)
    for q in fac:
        if pow(g, phi // q, p) == 1:
            return False
    return True

def smallest_primitive_root(p: int) -> int:
    """Return the smallest primitive root modulo prime p."""
    for g in range(2, p):
        if is_primitive_root(g, p):
            return g
    raise ValueError(f"No primitive root found for p={p}")
```

```python
from typing import Dict, Tuple

def build_iota_mu_k(p: int, k: int, g: int | None = None) -> Dict[int, int]:
    """Map elements of mu_k (subgroup of k-th roots) to indices 0..k-1."""
    if g is None:
        g = smallest_primitive_root(p)
    qk = pow(g, (p - 1) // k, p)  # generator of mu_k
```

```
    iota = {}
    x = 1
    for m in range(k):
        iota[x] = m
        x = (x * qk) % p
    # iota has size k
    return iota

def allowed_orders(p: int, k_max: int) -> list[int]:
    orders = []
    for k in range(2, k_max + 1):
        if (p - 1) % k == 0:
            orders.append(k)
    return orders
```

```
import torch

@torch.no_grad()
def inv_mod_p(x: torch.Tensor, p: int) -> torch.Tensor:

"""Modular inverse for x mod p (x nonzero). Uses Fermat little theorem since p
is prime."""
    # pow in python operates on ints, so use tensor exponentiation via modular
exponentiation loop
    # For small p and n, simplest is to do it on CPU ints; for speed, cache
inverses per p.
    x_int = x.to(torch.int64)
    inv = torch.empty_like(x_int)
    for idx in range(x_int.numel()):
        inv.view(-1)[idx] = pow(int(x_int.view(-1)[idx].item()), p - 2, p)
    return inv

@torch.no_grad()
def psi_normalize(M: torch.Tensor, row_center: bool = True, eps: float = 1e-8) -
> torch.Tensor:
    M = M - M.mean()
    M = M / (M.std(unbiased=False) + eps)
    if row_center:
        M = M - M.mean(dim=-1, keepdim=True)
    return M

@torch.no_grad()
def build_mask_phase(
    n: int,
    p: int,
```

```
    k: int,
    a_cos: float,
    a_sin: float = 0.0,
    b_inf: float = 0.0,
    b_zero: float = 0.0,
    device: str = "cpu",
    row_center: bool = True,
) -> torch.Tensor:
    """Build normalized mask M^{(k)} of shape (n,n) using multiplicative
coupling i*j^{-1}.

    Zero handling:
      - j==0 -> class inf (bias b_inf)
      - i==0 and j!=0 -> class zero (bias b_zero)
    """
    g = smallest_primitive_root(p)
    iota = build_iota_mu_k(p, k, g=g)

    # indices 0..n-1 as field elements
    I = torch.arange(n, device=device, dtype=torch.int64)
    J = torch.arange(n, device=device, dtype=torch.int64)
    ii = I[:, None].expand(n, n)
    jj = J[None, :].expand(n, n)

    M = torch.empty((n, n), device=device, dtype=torch.float64)

    # handle j==0
    j0 = (jj == 0)
    M[j0] = float(b_inf)

    # for j!=0, compute ratio i * inv(j)
    jnz = ~j0
    jj_nz = jj[jnz]
    inv_j = inv_mod_p(jj_nz, p)
    ratio = (ii[jnz] * inv_j) % p

    # handle i==0 separately (optional)
    i0 = (ii[jnz] == 0)
    if i0.any():
        M[jnz][i0] = float(b_zero)

    # for nonzero ratio in F_p^*, compute r_k(x)=x^{(p-1)/k} in mu_k
    # define mask only for x!=0 (ratio==0 already handled)
    nz = ~i0
    ratio_nz = ratio[nz]

    # r_k
    exp = (p - 1) // k
```

```
    r = torch.empty_like(ratio_nz)
    for idx in range(ratio_nz.numel()):
        r.view(-1)[idx] = pow(int(ratio_nz.view(-1)[idx].item()), exp, p)

    # map r to subgroup index m
    m = torch.empty_like(r)
    for idx in range(r.numel()):
        m.view(-1)[idx] = iota[int(r.view(-1)[idx].item())]

    # phase-coded bias
    theta = 2.0 * math.pi * (m.to(torch.float64) / float(k))
    bias = float(a_cos) * torch.cos(theta) + float(a_sin) * torch.sin(theta)

    tmp = M[jnz]
    tmp[nz] = bias
    M[jnz] = tmp

    return psi_normalize(M, row_center=row_center).to(torch.float32)
```

```
@torch.no_grad()
def row_mean_match(M_null: torch.Tensor, M_target: torch.Tensor) ->
torch.Tensor:
    """Affine row-mean matching: match per-row means exactly."""
    return M_null - M_null.mean(dim=-1, keepdim=True) + M_target.mean(dim=-1,
keepdim=True)

@torch.no_grad()
def frob_norm(M: torch.Tensor, eps: float = 1e-12) -> torch.Tensor:
    return torch.sqrt((M * M).sum() + eps)

@torch.no_grad()
def norm_match(M_null: torch.Tensor, M_target: torch.Tensor, eps: float = 1e-12)
-> torch.Tensor:
    s = frob_norm(M_target, eps=eps) / frob_norm(M_null, eps=eps)
    return M_null * s

@torch.no_grad()
def frob_corr(Ma: torch.Tensor, Mb: torch.Tensor, eps: float = 1e-12) -> float:
    num = float((Ma * Mb).sum().item())
    den = float(frob_norm(Ma, eps=eps).item() * frob_norm(Mb, eps=eps).item())
    return num / (den + 1e-12)
```

E. Soft-weighted null sampling (D3C-OM2): correlation weights + floor

```
import random

def soft_weights_from_corr(c_abs: list[float], w_min: float = 0.05) ->
list[float]:
    ws = [max(w_min, 1.0 / (1.0 + c)) for c in c_abs]
    z = sum(ws)
    return [w / z for w in ws]

@torch.no_grad()
def sample_null_order(
    k: int,
    orders: list[int],
    corr_abs: dict[tuple[int,int], float],
    rng: random.Random,
    w_min: float = 0.05,
) -> int:
    # candidate ell != k
    cands = [ell for ell in orders if ell != k]
    c_abs = [abs(corr_abs[(k, ell)]) for ell in cands]
    probs = soft_weights_from_corr(c_abs, w_min=w_min)
    r = rng.random()
    acc = 0.0
    for ell, p in zip(cands, probs):
        acc += p
        if r <= acc:
            return ell
    return cands[-1]
```

Note: corr_abs can be computed once per (p, k, ell) after building masks and applying the same ψ, row-mean matching, and norm matching.

F. Switchback probe (forward-only) skeleton

```
@torch.no_grad()
def switchback_probe(model, batch, mask_mode_struct, mask_mode_null, metric_fn):
    """Compute Δ_t = R(struct) - R(null) at fixed weights (no backward)."""
    # Configure model to use structured mask
    model.set_mask_mode(mask_mode_struct)
    out_s = model(**batch)
    R_s = metric_fn(out_s, batch)
```

```
    # Configure model to use null mask
    model.set_mask_mode(mask_mode_null)
    out_n = model(**batch)
    R_n = metric_fn(out_n, batch)

    return float(R_s - R_n)
```

G. EXP3 (sparse joint bandit) skeleton

```
import numpy as np

class EXP3:
    def __init__(self, n_arms: int, eta: float = 0.2, seed: int = 0):
        self.n = n_arms
        self.eta = eta
        self.rng = np.random.default_rng(seed)
        self.w = np.ones(n_arms, dtype=np.float64)

    def probs(self):
        wsum = self.w.sum()
        return self.w / wsum

    def sample(self):
        p = self.probs()
        a = int(self.rng.choice(self.n, p=p))
        return a, p[a]

    def update(self, arm: int, p_arm: float, reward: float):
        # reward should be clipped, e.g. in [-0.5, 0.5]
        # importance-weighted reward estimate
        rhat = reward / max(p_arm, 1e-12)
        self.w[arm] *= np.exp(self.eta * rhat)
```

H. Weighted median + block bootstrap CI

```
def weighted_median(values, weights):
    """Return weighted median. values, weights are 1D lists/arrays."""
    idx = np.argsort(values)
    v = np.asarray(values)[idx]
    w = np.asarray(weights)[idx]
```

```
    cw = np.cumsum(w) / np.sum(w)
    return float(v[np.searchsorted(cw, 0.5)])

def block_bootstrap_ci(values, weights, block_len=5, n_boot=2000, alpha=0.05,
seed=0):
    rng = np.random.default_rng(seed)
    T = len(values)
    idxs = np.arange(T)
    stats = []
    for _ in range(n_boot):
        # sample blocks with replacement
        chosen = []
        while len(chosen) < T:
            start = int(rng.integers(0, max(1, T - block_len + 1)))
            chosen.extend(list(range(start, min(T, start + block_len))))
        chosen = chosen[:T]
        v = [values[i] for i in chosen]
        w = [weights[i] for i in chosen]
        stats.append(weighted_median(v, w))
    stats = np.sort(np.array(stats))
    lo = float(stats[int((alpha/2) * n_boot)])
    hi = float(stats[int((1 - alpha/2) * n_boot) - 1])
    return lo, hi
```

Implementation notes (best practices)

1. **Cache masks** per (p, k, n) and per null type (N1/N2/N3). For p<1000 and k_max small, precompute on CPU in float64 and move to GPU as needed.
2. **Fixed evaluation slice** for switchback probes reduces batch noise and avoids reward hacking.
3. **Pre-specify**:
4. probe budget (max #probes)
5. post-warmup window $[t_0, t_1]$
6. primary endpoint and inference method
7. null ladder definitions
8. **Report sensitivity**:
9. uniform vs weighted null sampling
10. w_min floor
11. block length for bootstrap

Minimal 9-arm suggestion (for interpretability)

Let $\tau \in \{1.0, 0.5, 0.2\}$, $\beta \in \{0, 0.5, 1.0\}$. Define arms as all pairs (τ, β) (9). Use switchback probes (structured vs N3) outside the arm space to keep the controller simpler.

Instrument:

- posterior mass on $\beta = 0$
- how often controller prefers larger β on multiplicative tasks

---

References

The following works provide background, contrast, or partial inspiration for the components of CHAR-Adapt-R3-BD-JIS-R++ (D3C-OM2). None implement the same combination of **finite-field character masks**, **in-training causal switchbacks**, or **theory-matched null ladders**, but they situate the approach in existing literature.

1. Vaswani, A. et al. (2017). *Attention Is All You Need*. NeurIPS.

2. Baseline transformer attention; no explicit algebraic inductive bias.

3. Yun, C. et al. (2020). *Are Transformers Universal Approximators of Sequence-to-Sequence Functions?* ICLR.

4. Formal expressivity results motivating inductive biases for compositional generalization.

5. Cohen, T. & Welling, M. (2016). *Group Equivariant Convolutional Networks*. ICML.

6. Canonical reference for group-structured inductive biases; contrasts with character-based (rather than representation-based) biasing.

7. Kondor, R. & Trivedi, S. (2018). *On the Generalization of Equivariance and Convolution in Neural Networks*. ICML.

8. Theoretical background on symmetry and invariance in neural architectures.

9. Choromanski, K. et al. (2021). *Performer: Rethinking Attention with Favorable Properties*. ICLR.

10. Structured attention approximations; differs fundamentally from algebraic masking.

11. Su, J. et al. (2021). *RoFormer: Enhanced Transformer with Rotary Position Embedding*. arXiv.

12. Example of structured, but non-algebraic, attention bias (rotations, not finite fields).

13. Shalev-Shwartz, S. et al. (2012). *Online Learning and Online Convex Optimization*. Foundations and Trends in ML.

14. Background for EXP3/UCB-style bandit controllers used instrumentally in this work.

15. Bubeck, S. & Cesa-Bianchi, N. (2012). *Regret Analysis of Stochastic and Nonstochastic Multi-armed Bandit Problems*. Foundations and Trends in ML.

16. Formal regret guarantees motivating sparse joint-bandit control.

17. Efron, B. & Tibshirani, R. (1994). *An Introduction to the Bootstrap*. CRC Press.

18. Statistical foundation for bootstrap and block-bootstrap confidence intervals.

19. Lahiri, S. N. (2003). *Resampling Methods for Dependent Data*. Springer.

    ◦ Justification for block bootstrap under temporal dependence in switchback probes.

20. Tao, T. & Vu, V. (2012). *Random matrices: universality of local eigenvalue statistics*. Acta Mathematica.

    ◦ Conceptual background for spectral diagnostics (used here instrumentally, not normatively).

21. Serre, J.-P. (1997). *Lectures on $N_x(p)$*. CRC Press.

    ◦ Reference for character orthogonality and distributions motivating theory-matched nulls.

22. Popper, K. (1959). *The Logic of Scientific Discovery*. Routledge.

    ◦ Philosophical grounding for falsification-first experimental design and null ladders.

What “success” looks like

Primary endpoint (per seed): weighted median T of $\Delta_t^{(N3)}$ in $[t_0, t_1]$ has CI excluding 0 on multiplicative tasks.

Secondary:

- effect decreases N1→N2→N3
- Corr(controller preference, EMA(\Delta_t)) > 0 with CI excluding 0
- cross-prime stability of k-selection (KL small)